Krish Karanikar 241210065 CSE 1

Title: Investigating addressing Modes using C code

Theory:

Addressing modes are techniques used by the CPU to identify the location of the operand(s) needed for executing an instruction. They provide rules for interpreting the address field in an instruction, helping the CPU fetch operands correctly.

The types of addressing modes includes

- Implicit addressing : The instruction does not mention the operand directly. The CPU knows what to use from the instruction itself, usually a special register like the accumulator or the stack. It is used for special instructions or control commands like CLA, PUSH, and RET, where the operand is automatically known from the instruction itself
- Immediate addressing: The operand is the part of the instruction itself. It is used when the value is known while writing the program
- Direct addressing: The instruction contains the memory address of the operand. The CPU accesses the data directly from that address.
- Indirect addressing: The instruction contains the address of a memory location, which itself stores the actual address of the operand. The CPU first accesses this memory location to get the effective address, and then fetches the operand.
- Register addressing: The operand is located in a CPU register specified by the instruction.
- Register indirect addressing: The register specified in the instruction contains the memory address of the operand.
- Displacement addressing: The operand's effective address is calculated by adding a constant value (displacement) to the contents of one or more registers.
- Stack addressing: The operand is implicitly taken from the top of the stack, without being mentioned in the instruction

Implementation:

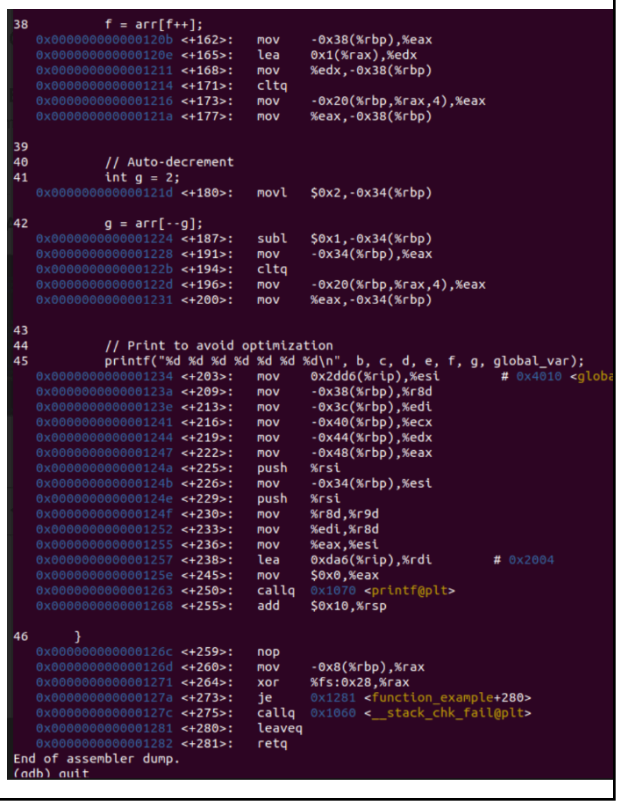
```
6       void function_example() {
   0x0000000000001169 <+0>:     endbr64
   0x000000000000116d <+4>:     push   %rbp
   0x000000000000116e <+5>:     mov    %rsp,%rbp
   0x0000000000001171 <+8>:     sub    $0x60,%rsp
   0x0000000000001175 <+12>:    mov    %fs:0x28,%rax
   0x000000000000117e <+21>:    mov    %rax,-0x8(%rbp)
   0x0000000000001182 <+25>:    xor    %eax,%eax

7           // Stack Addressing (local variables)
8           int x = 5;
   0x0000000000001184 <+27>:    movl   $0x5,-0x54(%rbp)

9           int y = 10;
   0x000000000000118b <+34>:    movl   $0xa,-0x4c(%rbp)

10
11          // Immediate Addressing
12          int a = 20;
   0x0000000000001192 <+41>:    movl   $0x14,-0x58(%rbp)

13
14          // Register Direct (normal variable use)
15          int b = a;
   0x0000000000001199 <+48>:    mov    -0x58(%rbp),%eax
   0x000000000000119c <+51>:    mov    %eax,-0x48(%rbp)

16
17          // Indirect Addressing
18          int *p = &a;
   0x000000000000119f <+54>:    lea    -0x58(%rbp),%rax
   0x00000000000011a3 <+58>:    mov    %rax,-0x30(%rbp)
```

GDB Output

```c
#include <stdio.h>
// Direct Addressing (global variable)
int global_var = 50;

void function_example() {
    // Stack Addressing (local variables)
    int x = 5;
    int y = 10;

    // Immediate Addressing
    int a = 20;

    // Register Direct (normal variable use)
    int b = a;

    // Indirect Addressing
    int *p = &a;

    // Register Indirect
    int c = *p;

    // Indexed Addressing
    int arr[5] = {1,2,3,4,5};
    int d = arr[2];

    // Base Register Addressing (array + pointer)
    int *base = arr;
    int e = *(base + 3);

    // Relative Addressing (loop / control flow)
    for(int i = 0; i < 3; i++) {
        x += i;
    }

    // Auto-increment
    int f = 0;
    f = arr[f++];

    // Auto-decrement
    int g = 2;
    g = arr[--g];

    // Print to avoid optimization
    printf("%d %d %d %d %d %d %d\n", b, c, d, e, f, g, global_var);
}

int main() {
    function_example();
    return 0;
}
```

Used Code

```
22
23          // Indexed Addressing
24          int arr[5] = {1,2,3,4,5};
   0x00000000000011b0 <+71>:    movl   $0x1,-0x20(%rbp)
   0x00000000000011b7 <+78>:    movl   $0x2,-0x1c(%rbp)
   0x00000000000011be <+85>:    movl   $0x3,-0x18(%rbp)
   0x00000000000011c5 <+92>:    movl   $0x4,-0x14(%rbp)
   0x00000000000011cc <+99>:    movl   $0x5,-0x10(%rbp)

25          int d = arr[2];
   0x00000000000011d3 <+106>:   mov    -0x18(%rbp),%eax
   0x00000000000011d6 <+109>:   mov    %eax,-0x40(%rbp)
--Type <RET> for more, q to quit, c to continue without paging--

26
27          // Base Register Addressing (array + pointer)
28          int *base = arr;
   0x00000000000011d9 <+112>:   lea    -0x20(%rbp),%rax
   0x00000000000011dd <+116>:   mov    %rax,-0x28(%rbp)

29          int e = *(base + 3);
   0x00000000000011e1 <+120>:   mov    -0x28(%rbp),%rax
   0x00000000000011e5 <+124>:   mov    0xc(%rax),%eax
   0x00000000000011e8 <+127>:   mov    %eax,-0x3c(%rbp)

30
31          // Relative Addressing (loop / control flow)
32          for(int i = 0; i < 3; i++) {
   0x00000000000011eb <+130>:   movl   $0x0,-0x50(%rbp)
   0x00000000000011f2 <+137>:   jmp    0x11fe <function_example+

33              x += i;
   0x00000000000011f4 <+139>:   mov    -0x50(%rbp),%eax
   0x00000000000011f7 <+142>:   add    %eax,-0x54(%rbp)

32          for(int i = 0; i < 3; i++) {
   0x00000000000011fa <+145>:   addl   $0x1,-0x50(%rbp)
   0x00000000000011fe <+149>:   cmpl   $0x2,-0x50(%rbp)
   0x0000000000001202 <+153>:   jle    0x11f4 <function_example+
```



```
38          f = arr[f++];
   0x000000000000120b <+162>:   mov    -0x38(%rbp),%eax
   0x000000000000120e <+165>:   lea    0x1(%rax),%edx
   0x0000000000001211 <+168>:   mov    %edx,-0x38(%rbp)
   0x0000000000001214 <+171>:   cltq
   0x0000000000001216 <+173>:   mov    -0x20(%rbp,%rax,4),%eax
   0x000000000000121a <+177>:   mov    %eax,-0x38(%rbp)

39
40          // Auto-decrement
41          int g = 2;
   0x000000000000121d <+180>:   movl   $0x2,-0x34(%rbp)

42          g = arr[--g];
   0x0000000000001224 <+187>:   subl   $0x1,-0x34(%rbp)
   0x0000000000001228 <+191>:   mov    -0x34(%rbp),%eax
   0x000000000000122b <+194>:   cltq
   0x000000000000122d <+196>:   mov    -0x20(%rbp,%rax,4),%eax
   0x0000000000001231 <+200>:   mov    %eax,-0x34(%rbp)

43
44          // Print to avoid optimization
45          printf("%d %d %d %d %d %d %d\n", b, c, d, e, f, g, global_var);
   0x0000000000001234 <+203>:   mov    0x2dd6(%rip),%esi        # 0x4010 <globa
   0x000000000000123a <+209>:   mov    -0x38(%rbp),%r8d
   0x000000000000123e <+213>:   mov    -0x3c(%rbp),%edi
   0x0000000000001241 <+216>:   mov    -0x40(%rbp),%ecx
   0x0000000000001244 <+219>:   mov    -0x44(%rbp),%edx
   0x0000000000001247 <+222>:   mov    -0x48(%rbp),%eax
   0x000000000000124a <+225>:   push   %rsi
   0x000000000000124b <+226>:   mov    -0x34(%rbp),%esi
   0x000000000000124e <+229>:   push   %rsi
   0x000000000000124f <+230>:   mov    %r8d,%r9d
   0x0000000000001252 <+233>:   mov    %edi,%r8d
   0x0000000000001255 <+236>:   mov    %eax,%esi
   0x0000000000001257 <+238>:   lea    0xda6(%rip),%rdi        # 0x2004
   0x000000000000125e <+245>:   mov    $0x0,%eax
   0x0000000000001263 <+250>:   callq  0x1070 <printf@plt>
   0x0000000000001268 <+255>:   add    $0x10,%rsp

46      }
   0x000000000000126c <+259>:   nop
   0x000000000000126d <+260>:   mov    -0x8(%rbp),%rax
   0x0000000000001271 <+264>:   xor    %fs:0x28,%rax
   0x000000000000127a <+273>:   je     0x1281 <function_example+280>
   0x000000000000127c <+275>:   callq  0x1060 <__stack_chk_fail@plt>
   0x0000000000001281 <+280>:   leaveq
   0x0000000000001282 <+281>:   retq
End of assembler dump.
(gdb) quit
```

Conclusion

Successfully explored all addressing modes